

Data Structures Assignment

Array, Stack, and Queue — Menu-Driven Programs

Assignment Policy

- This assignment has been intentionally kept simple so that every student is able to complete it and can focus on genuinely understanding data structures rather than struggling with complexity.
- If you get stuck, you are encouraged to seek help from your course slides, classmates, or the internet — but strictly for the purpose of understanding concepts, not for obtaining ready-made solutions.
- The use of AI tools to generate solutions, or the copying of assignments from any source, is strictly prohibited. All submitted work must be your own.
- Your viva will conduct in your first lab class.

General Instructions

- **Implement all programs in Java.**
 - Solve **all 5 questions** (Array, Stack, Queue, Real-World Examples, and Advantages/Disadvantages).
 - Use **a single array/variable** to store your data structure (no built-in library structures like `ArrayList`, `Stack` class, `Queue` class, `list.append()`-based dynamic tricks, etc. — implement the logic yourself).
 - Each program must be **menu-driven** (a loop that keeps showing options until the user chooses Exit).
 - You may implement each solution **either using a class** (OOP style, with the array and size/top/front/rear as class members) **or without a class** (plain functions/procedures operating on a global or passed-in array). Choose whichever style your course requires, or do both for practice.
 - Assume a **fixed maximum size** for the array/stack/queue (e.g., 10 or 20), decided by you at the start of the program.
 - You can use `int` data type
 - Add simple validation (e.g., check for overflow, underflow, and invalid index) and print clear messages for each operation.
-

Question 1 — Array Operations

Scenario: A small library wants a simple program to manage the list of book IDs currently on a shelf. The shelf can hold a limited number of books at a time. The librarian needs a menu to manage this list from the console.

Also write advantages and disadvantages of array data structure.

Requirements: Implement an Array-based program that supports the following operations through a menu:

1. **Add** a new value to the end of the array
2. **Insert** a value at a specific index (given by the user), shifting existing elements to make room
3. **Fill** the array with values (read multiple values from the user until the array is full or the user stops)
4. **Delete** the last element from the array
5. **Delete by index** (given by the user), shifting remaining elements to fill the gap
6. **Display** all elements currently in the array
7. **Search** for a value (return whether it exists and at which index)
8. **Get** the value at a specific index (given by the user)
9. **Replace / Update** the value at a specific index with a new value given by the user
10. **Size** show the current number of elements in the array
11. **Exit**

Sample Menu:

```
----- ARRAY MENU -----
1. Add value
2. Insert at index
3. Fill array
4. Delete last element
5. Delete by index
6. Display
7. Search value
8. Get value at index
9. Replace/Update value at index
10. size
11. Exit
Enter your choice:
```

Question 2 — Stack Operations

Scenario: You have a pile of plates (whole numbers written on each plate) stacked one on top of another. You can only add a new plate on top, and you can only remove the plate that is currently on top. Build a program to simulate this stack of integer values.

Requirements: Implement a Stack-based program (using a single array and a `top` variable) that supports the following operations through a menu:

1. **Push** — add a new action/value to the top of the stack
2. **Pop** — remove and show the value from the top of the stack
3. **Display** — show all values currently in the stack (top to bottom or bottom to top)

4. **Size** — show the current number of elements in the stack
5. **isEmpty** — check whether the stack is empty
6. **isFull** — check whether the stack has reached its maximum capacity
7. **Exit**

Sample Menu:

```
----- STACK MENU -----
1. Push
2. Pop
3. Display
4. Size
5. isEmpty
6. isFull
7. Exit
Enter your choice:
```

Question 3 — Queue Operations

Scenario: A small bank counter serves customers using token numbers (integers). Every arriving customer is given a token number, which joins the back of the queue, and the counter always serves the token at the front of the queue first. Build a program to manage this queue of integer token numbers.

Requirements: Implement a Queue-based program (using a single array and `front/rear` variables) that supports the following operations through a menu:

1. **Enqueue** — add a new print job/value to the back of the queue
2. **Dequeue** — remove and show the value from the front of the queue
3. **Display** — show all values currently in the queue from front to rear
4. **Size** — show the current number of elements in the queue
5. **isEmpty** — check whether the queue is empty
6. **isFull** — check whether the queue has reached its maximum capacity
7. **Exit**

Sample Menu:

```
----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Size
5. isEmpty
6. isFull
7. Exit
Enter your choice:
```

Question 4 — Real-World Examples

Task: For each data structure write, 5 **real-world (or software) examples** where that data structure is a natural fit. For every example, write **one line explaining why** that data structure suits the scenario. Also write their advantages and disadvantages.

Question 5 — Advantages and Disadvantages

Task: Write the **advantages and disadvantages** of each of the following data structures. Write at least **2–3 points** for each, in your own words, based on what you observed while completing Questions 1 to 3.